COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 / Data Structures and Algorithms | **SECTION:** | CPE21S3 |
| **DATE PERFORMED:** | September 8, 2026 | **DATE SUBMITTED:** | September 10, 2026 |
| **NAME:** | Reyes, Fenzie Aaron A. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 7.1 SORTING ALGORITHMS PT1

### 1. PROCEDURE OUTPUT

What procedures did you perform?

**Code + Console Screenshot**

sorting.h | main.cpp

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

const int max_size = 100;

int main() {
    int testArray[max_size];

    // Separate arrays for each sorting algorithm
    int bubbleArray[max_size];
    int selectionArray[max_size];
    int insertionArray[max_size];
    srand(time(0));

    // Fill the array with 100 random numbers
    for (int i = 0; i < max_size; i++) {

        testArray[i] = rand() % 100 + 1;
        // Copy original values to each sorting array
        bubbleArray[i] = testArray[i];
        selectionArray[i] = testArray[i];
        insertionArray[i] = testArray[i];
    }

    // Display original array
    std::cout << "Array elements:\n";
    displayArray(testArray, max_size);
```

sorting.h main.cpp

sorting.h > bubbleSort<T>(T [], size_t)

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <iostream>
#include <utility>


// DISPLAYING ALL ARRAY ELEMENTS
void displayArray(const int values[], int maxSize) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << values[i] << " ";
        // Move to the next line after every 10 elements
        if ((i + 1) % 10 == 0) {
            std::cout << "\n";
        }
    }
}
```

```
' '--dbgExe=C:\msys64\ucrt64\bin\gdb.exe' '--interpreter=
Array elements:
50 84 71 70 55 52 58 61 95 65
77 84 79 6 50 32 22 81 58 47
75 13 30 41 6 6 14 63 56 87
20 55 11 8 75 92 37 88 62 27
80 20 3 35 26 66 4 5 86 73
82 91 79 89 64 78 19 41 69 65
78 7 68 8 36 76 44 57 45 17
16 5 27 9 56 86 45 40 37 80
94 39 34 70 60 21 47 42 82 98
14 52 48 94 72 50 41 28 53 96
```

| | |
|---|---|
| **Observations** | The problem generated 100 random integer values from 1 to 100. It uses a for loop to go through each element and prints values sequentially. The code also formats the output by moving to a new line after every 10 elements, making the 100 generated values easier to read and check before applying a sorting algorithm. |

*Table 7-1. Array of Values for Sort Algorithm Testing*

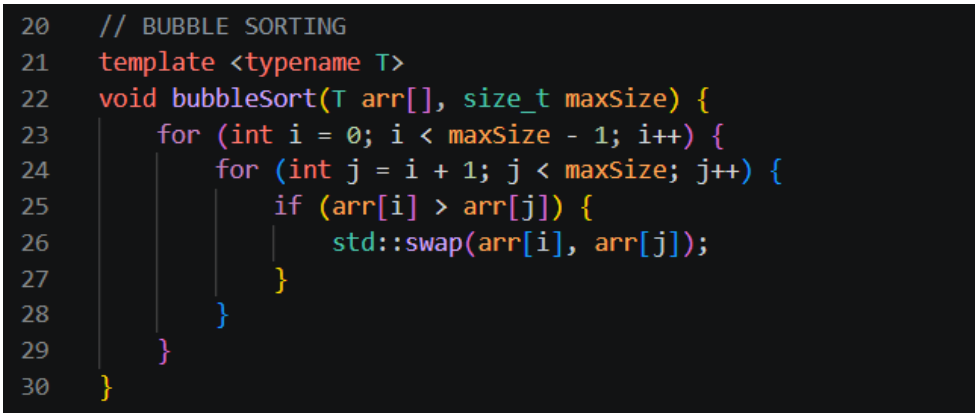
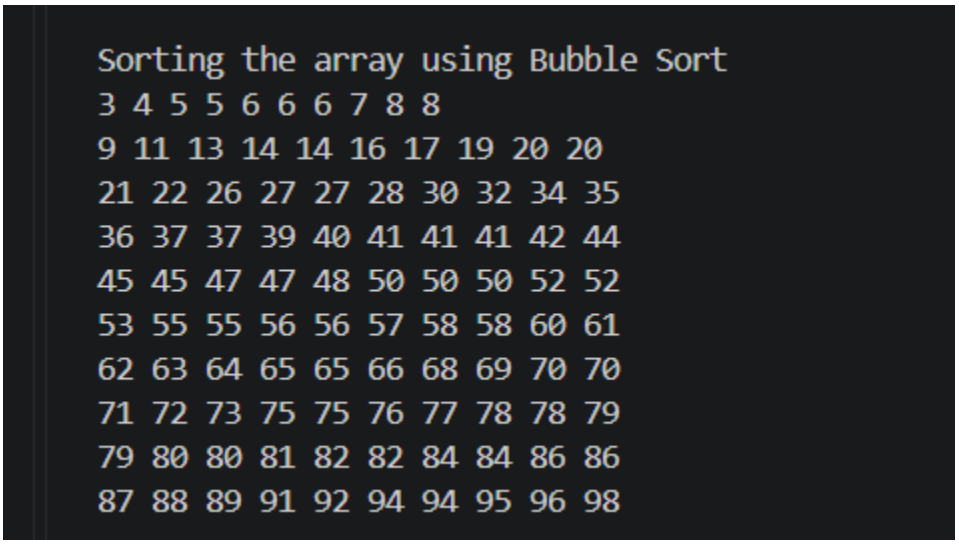
| | |
|---|---|
| **Code + Console Screenshot** |  <br>  |
| **Observations** | Bubble sort in this case arranged the array elements in ascending order, starting from the smallest value and ending with the largest value. From what we have seen in this output, the algorithm repeatedly compares elements and swaps them when they are in the wrong order. After that process, the output shows that all 100 values were successfully arranged in accordance to how bubble sort works. |

Table 7-2. Bubble Sort Technique

**Code + Console Screenshot**

```cpp
// FIND THE POSITION OF THE SMALLEST ELEMENT
template <typename T>
int routine_smallest(T arr[], int k, const int maxSize) {
    int position;
    position = k;
    // Find the smallest element
    for (int j = k + 1; j < maxSize; j++) {
        if (arr[j] < arr[position]) {
            position = j;
        }
    }
    // Return the position of the smallest element
    return position;
}


// SELECTION SORTING
template <typename T>
void selectionSort(T arr[], int maxSize) {
    int minVal;
    for (int i = 0; i < maxSize - 1; i++) {
        minVal = routine_smallest(arr, i, maxSize);
        std::swap(arr[i], arr[minVal]);
    }
}
```

```
Sorting the array using Selection Sort
3 4 5 5 6 6 6 7 8 8
9 11 13 14 14 16 17 19 20 20
21 22 26 27 27 28 30 32 34 35
36 37 37 39 40 41 41 41 42 44
45 45 47 47 48 50 50 50 52 52
53 55 55 56 56 57 58 58 60 61
62 63 64 65 65 66 68 69 70 70
71 72 73 75 75 76 77 78 78 79
79 80 80 81 82 82 84 84 86 86
87 88 89 91 92 94 94 95 96 98
```

**Observations**

Selection Sort Algorithm is a sorting technique in which it repeatedly finds the smallest remaining element and places it in the correct position. The sorted result was in ascending order in this case, although the process used a different approach from bubble sort, it still produced the same correctly sorted array.

Table 7-3. Selection Sort Algorithm

**Code + Console Screenshot**

```
60  // INSERTION SORTING
61  template <typename T>
62  void insertionSort(T arr[], int maxSize) {
63      int k, j;
64      T temp;
65      // Swapping, cannot use std::swap
66      k = 1;
67      while (k < maxSize) {
68          temp = arr[k];
69          j = k - 1;
70          while (j >= 0 && temp < arr[j]) {
71              arr[j + 1] = arr[j];
72              j--; // Backtracking part
73          }
74          // Reassigning values and moving to next element
75          arr[j + 1] = temp;
76          k++;
77      }
78  }
79
80  #endif
```

```
Sorting the array using Insertion Sort
3 4 5 5 6 6 6 7 8 8
9 11 13 14 14 16 17 19 20 20
21 22 26 27 27 28 30 32 34 35
36 37 37 39 40 41 41 41 42 44
45 45 47 47 48 50 50 50 52 52
53 55 55 56 56 57 58 58 60 61
62 63 64 65 65 66 68 69 70 70
71 72 73 75 75 76 77 78 78 79
79 80 80 81 82 82 84 84 86 86
87 88 89 91 92 94 94 95 96 98
```

**Observations**

Insertion Sort arranged values by taking each element and inserting it into its correct position among the previously sorted elements. Larger elements were shifted to the right to make space for the current value. The final output of this console showed the same ascending order as other techniques used previously.

Table 7-4. Insertion Sort Algorithm

COMPUTER ENGINEERING

QUEZON CITY

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

**Code + Console Screenshot**

main.cpp

main.cpp > CountVotes<T>(T [], int, T)

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

const int maxSize = 100;

// ============================
// FIND SMALLEST
// ============================
template <typename T>
int routineSmallest(T arr[], int K, int maxSize){
    int small = K;
    for (int i = K + 1; i < maxSize; i++){
        if (arr[i] < arr[small])
            small = i;
    }
    return small;
}

// ============================
// SELECTION SORT
// ============================
template <typename T>
void selectionSort(T arr[], int maxSize){
    for (int i = 0; i < maxSize - 1; i++){
        int small = routineSmallest(arr, i, maxSize);
        T temp = arr[i];
        arr[i] = arr[small];
        arr[small] = temp;
    }
}
```

```cpp
// ============================
// COUNT VOTES
// ============================
template <typename T>
int CountVotes(T arr[], int maxSize, T candidate){
    int count = 0;
    for (int i = 0; i < maxSize; i++){
        if (arr[i] == candidate)
            count++;
    }
    return count;
}
```

main.cpp

main.cpp > CountVotes<T>(T [], int, T)

```cpp
// ============================
// MAIN
// ============================
int main(){
    int arr[maxSize];

    string candidates[5] =
    {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustín",
        "Deja Jayla Bañaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    srand(time(0));
    // Generate random votes from 1 to 5
    for (int i = 0; i < maxSize; i++)
        arr[i] = (rand() % 5) + 1;


    // Sort array
    selectionSort(arr, maxSize);

    // Display sorted array
    cout << "SORTED ARRAY:" << endl;

    for (int i = 0; i < maxSize; i++){
        cout << arr[i] << " ";
        if ((i + 1) % 20 == 0)
            cout << endl;
    }
```

main.cpp

main.cpp > main()

```cpp
int main(){
    // Count votes
    int votes[5];
    cout << endl;
    cout << "VOTE RESULTS:" << endl;

    for (int i = 0; i < 5; i++){
        votes[i] = CountVotes(arr, maxSize, i + 1);
        cout << "Candidate " << i + 1
             << " - " << candidates[i]
             << ": " << votes[i]
             << " votes" << endl;
    }

    // Find winner
    int winner = 0;
    for (int i = 1; i < 5; i++){
        if (votes[i] > votes[winner])
            winner = i;
    }

    // Display winner
    cout << endl;
    cout << "============================" << endl;
    cout << "WINNING CANDIDATE" << endl;
    cout << "============================" << endl;
    cout << "Candidate " << winner + 1
         << ": " << candidates[winner] << endl;
    cout << "Total Votes: "
         << votes[winner] << endl;

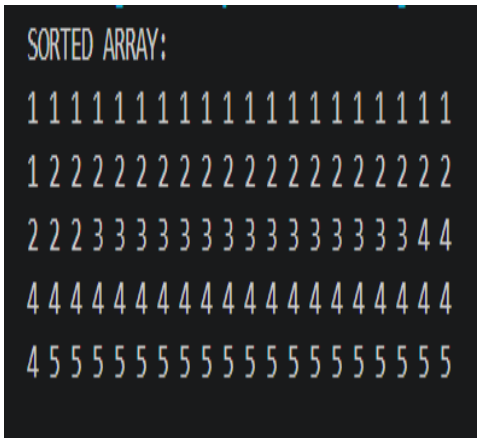
    return 0;
}
```

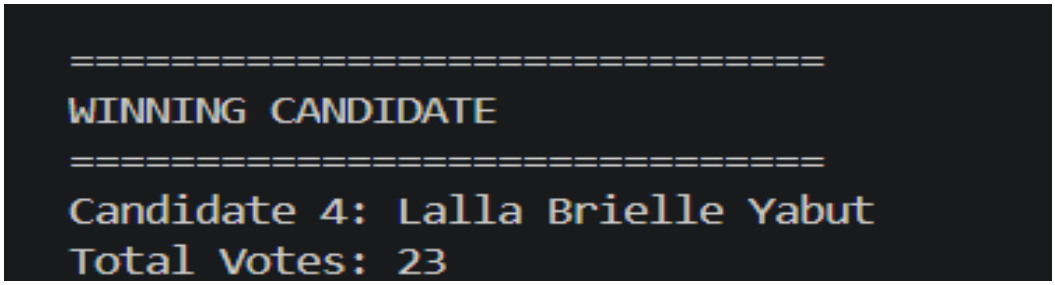
```
SORTED ARRAY:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

VOTE RESULTS:
Candidate 1 - Bo Dalton Capistrano: 21 votes
Candidate 2 - Cornelius Raymon Agustín: 22 votes
Candidate 3 - Deja Jayla Bañaga: 15 votes
Candidate 4 - Lalla Brielle Yabut: 23 votes
Candidate 5 - Franklin Relano Castro: 19 votes

============================
WINNING CANDIDATE
============================
Candidate 4: Lalla Brielle Yabut
Total Votes: 23
```

| | |
|---|---|
| **Observations** | The program generates 100 random votes for five candidates using values ranging from 1 to 5. With this, I use Selection Sort to arrange the votes in ascending order, followed by the CountVotes() function to count the votes received by each candidate. The candidate with the highest number of votes is then displayed and identified as the winner. In this code, I chose Selection Sort because it is simple to implement and is suitable for the size of the array used in this activity. Furthermore, the selection sort algorithm in this code has a time complexity of $O(n^2)$, while CountVotes() has a time complexity of O(n) for each candidate. Overall, the time complexity of the program is $O(n^2)$ |

| Output Console Showing Sorted Array | Manual Count | Count Result of Algorithm |
|---|---|---|
|  | 21 |  |
| | 22 |  |
| | 15 |  |
| | 23 |  |
| | 19 |  |




**Question:** Was your developed vote counting algorithm effective? Why or why not?

Yes, the algorithm used in this activity was effective because it correctly sorts and counts the votes for all five candidates. Although Selection Sort has $O(n^2)$ time complexity, it is suitable for 100 votes used in this activity.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** ChatGPT and Grammarly

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** To fix errors and better compose each observation and question.

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** The sole purpose of AI is only to guide me and correct me in my code errors.

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.